🎙️ SPEECH D'ENTRAÎNEMENT ORAL — BLOC 2 : ARCHITECTURE DATA & IA STRIPE

Certification Architecte en Intelligence Artificielle (RNCP 41993 — Niveau 7)
Durée du Speech : 5 Minutes Chrono *(suivi de 15 min de Q&A)*
Support projeté : `PRESENTATION_STRIPE_BLOC2.pptx` *(6 slides)* — 100 % synchronisé avec les 9 livrables PDF (`00_Executive_Summary.pdf` à `08_SQL_NoSQL_Queries.pdf` + schémas DBML `stripe_oltp` et `stripe_olap`). **100 % Conception théorique (aucun déploiement réel exigé).**
Posture : Tu es le **Lead Data & AI Architect** de Stripe. Tu présentes l'architecture Cloud AWS souveraine (`eu-west-3` Paris) dimensionnée pour **15 millions de transactions/jour** et un scoring IA **Stripe Radar** `< 100 ms` .

⏱️ RÉCAPITULATIF DU CHRONO (5 MINUTES — SLIDES 1 À 6)

Chrono	Slide (`PRESENTATION_STRIPE_BLOC2.pptx`)	Livrables Associés	Briques & Chiffres Clés	Compétence RNCP
00:00 – 00:55	**Slides 1 & 2** : *Convergence des 3 Mondes & Archi AWS*	Livrables `00` & `01`	OLTP ACID (`< 15 ms`), OLAP BI, NoSQL Télémétrie + IA **Stripe Radar** `< 100 ms` sur AWS Paris (`eu-west-3`)	**C1 & C2** (Collecte & Architecture)
00:55 – 02:15	**Slide 3** : *Modélisation Polyglotte & Pipeline CDC*	Livrables `02` à `05`	**Aurora PostgreSQL** (8 tables 3NF, `BIGINT` centimes) ➔ **Debezium CDC (WAL) + MSK ➔ Redshift Serverless** (3 faits, 6 dim) + **DocumentDB** (3 collections, TTL 90j) + **S3 Parquet / Airflow / dbt**	**C2 & C3** (Modélisation SQL/NoSQL & Flux)
02:15 – 03:20	**Slide 4** : *Chemin Critique IA Radar (`< 100 ms`) & MLOps*	Livrable `07`	Budget **95 ms p99** : Edge/WAF (**15 ms**) + **Feast Redis** (**20 ms**) + **OpenSearch k-NN** (**15 ms**) + **XGBoost ONNX CPU** `c6i` (**30 ms**) + Règles (**15 ms**) + **Shadow 48h & PSI Evidently (> 0.20)**	**C3** (Intégration ML Temps Réel)
03:20 – 04:15	**Slide 5** : *Haute Dispo (`RPO=0`) & Sécurité PCI-DSS / RGPD*	Livrable `06`	Multi-AZ Quorum 4/6 (**RPO = 0, RTO < 60s**), **Tokenization Vault (CloudHSM)**, **AES-256 KMS CMK**, **TLS 1.3**, **SHA-256 + Purge Airflow/Spark**	**C4** (Sécurité & Conformité)
04:15 – 05:00	**Slide 6** : *Requêtes SQL/NoSQL, FinOps & GreenOps*	Livrables `06` & `08`	4 requêtes SQL (GPV, Refus, Chargebacks 1%, Cohortes) + 3 agrégations NoSQL ; Budget **~25 000 $/mois** (Savings Plans -38%, Spot GPU -70%, GreenOps Paris 50g CO_2/kWh)	**C3 & C4** (Requêtes & FinOps)

🗣️ TEXTE À LIRE À L'ORAL (STYLE PROMPTEUR FLUIDE)

🌐 SLIDES 1 & 2 — Introduction, Les 3 Mondes de Données & Architecture AWS *(⏱️ 00:00 – 00:55 | 55 sec)*

- **Support à l'écran :** *Slide 1 puis Slide 2 (`PRESENTATION_STRIPE_BLOC2.pptx` — Livrables `00` & `01`)*

> « Bonjour à tous. Pour ce Bloc 2 consacré à l'architecture de données, je vous présente la conception de l'infrastructure Data et IA de **Stripe**, dimensionnée pour traiter **15 millions de**

> **transactions par jour** dans plus de 135 pays tout en propulsant notre moteur anti-fraude **Stripe Radar**.
>
> **[Passer à la Slide 2]** Comme illustré sur la Slide 2 (et dans mes livrables 00 et 01), notre architecture fait converger **trois mondes de données** aux exigences opposées au sein de la région souveraine **AWS Paris (`eu-west-3`)** :
>
> 1. **Le monde Transactionnel (OLTP) :** encaisser les paiements avec une disponibilité de **99,999 %**, une garantie **ACID absolue**, zéro double débit grâce à l'idempotence et une écriture en moins de 15 millisecondes.
> 2. **Le monde Analytique (OLAP) :** agréger des centaines de millions de lignes pour le reporting financier, le calcul du chiffre d'affaires et le suivi des litiges (*chargebacks*).
> 3. **Le monde Non Structuré et Télémétrie (NoSQL) :** absorber à très haute cadence le *clickstream*, les empreintes matérielles (*device fingerprints*) et les logs de webhooks JSON.
>
> Le tout sous une contrainte critique non négociable : **scorer chaque transaction entrante par IA en moins de 100 millisecondes**. »

- 👉 **[Passer à la Slide 3]**

🗄️ SLIDE 3 — 2. Modélisation Polyglotte (OLTP 3NF, OLAP Star Schema, NoSQL) & Pipeline CDC *(⏱️ 00:55 – 02:15 | 1 min 20)*

- **Support à l'écran :** *Slide 3 (`PRESENTATION_STRIPE_BLOC2.pptx` — Livrables `02`, `03`, `04` & `05`)*

> « Sur la **Slide 3**, pour éviter qu'une base unique ne s'écroule sous ces charges contradictoires, j'ai conçu une **persistance polyglotte** :
>
> - **Côté OLTP (`02_OLTP_ERD`) :** Le cœur financier repose sur **Amazon Aurora PostgreSQL Multi-AZ** modélisé en **3e Forme Normale (3NF)** autour de **8 tables** (`Merchant`, `Customer`, `PaymentMethod`, `Transaction`, `FraudEvaluation`, `Refund`, `Dispute` et `MerchantPayout`). Deux choix techniques majeurs y figurent : tous les montants sont stockés en entiers `BIGINT` **en centimes** pour éliminer toute erreur d'arrondi en virgule flottante, et un index unique sur `idempotency_key` empêche mathématiquement tout double débit en cas de coupure réseau.
> - **Côté OLAP (`03_OLAP_Schema`) :** Pour les requêtes analytiques massives sur **Amazon Redshift Serverless**, j'ai conçu un **Schéma en Étoile (*Star Schema*)** composé de **3 tables de faits** — `FactTransactions` (au grain de la transaction), `FactDailyMerchantRevenue` (pré-agrégée par marchand et par jour) et `FactFraudAlerts` — entourées de **6 dimensions** (`DimDate`, `DimMerchant`, `DimCustomer`, `DimPaymentMethod`, `DimGeoLocation`, `DimStatus`) utilisant des clés de substitution entières (*Surrogate Keys*) et une historisation **SCD Type 2**.
> - **Côté NoSQL (`04_NoSQL_Model`) :** Sur **Amazon DocumentDB (compatible MongoDB)**, j'ai modélisé **3 collections JSON** : `device_fingerprints` (qui imbrique en *embedding* la télémétrie matérielle pour une lecture en $O(1)$), `clickstream_events` (dynamique de frappe et mouvements de souris) et `webhook_delivery_logs`, protégée par un **index TTL de purge automatique à 90 jours**.
> - **Enfin, comment relier l'OLTP à l'OLAP sans ralentir les paiements (`05_Data_Pipeline`) ?** Nous interdisons tout `SELECT` analytique sur Aurora : **Debezium CDC** lit en continu le journal binaire (**WAL**) de PostgreSQL et pousse les événements dans **Amazon MSK (Apache Kafka)**, où **Apache Flink** calcule les agrégats sur fenêtres glissantes vers Redis, tandis que les connecteurs Kafka alimentent **Redshift** et notre **Data Lake S3 (Bronze / Silver / Gold en**

> **Parquet)** consolidé chaque nuit à 02h00 UTC par **Apache Airflow (MWAA), dbt et Great Expectations**. »

- 🛡️ *Raccord 100 % Slide 3 & Livrables 02 à 05.*
- 👉 **[Passer à la Slide 4]**

⚡ SLIDE 4 — 3. Le Chemin Critique d'Inférence IA : Stripe Radar en < 100 ms *(⏱ 02:15 – 03:20 | 1 min 05)*

- **Support à l'écran :** *Slide 4 (* `PRESENTATION_STRIPE_BLOC2.pptx` *— Livrable* `07` *)*

> « Voyons sur la **Slide 4** comment **Stripe Radar** respecte le budget strict de **moins de 100 millisecondes au 99e percentile (p99)** :
>
> Pour garantir une parité parfaite entre l'entraînement et la production (*zéro Training-Serving Skew*), nous utilisons le Feature Store **Feast**, avec un **Offline Store sur S3** et un **Online Store en mémoire vive sur Redis ElastiCache**.
>
> Lorsqu'un paiement arrive, notre budget de latence mesuré est de **95 millisecondes au p99**, décomposé en 5 étapes synchrones :
>
> 1. **15 ms** pour le routage DNS Route 53, le filtrage anti-DDoS AWS WAF et la terminaison TLS 1.3 sur l'ALB.
> 2. **20 ms** pour l'extraction multi-clés des features de vélocité temps réel depuis **Feast sur Redis ElastiCache** (lecture mémoire vive `< 5 ms`).
> 3. **15 ms** pour la recherche de proximité vectorielle **k-NN dans Amazon OpenSearch** afin de comparer la transaction aux clusters de fraude des dernières 48 heures.
> 4. **30 ms** pour l'inférence du modèle **XGBoost / LightGBM compilé en ONNX Runtime**, exécuté sur les processeurs **CPU** `c6i` **(instructions AVX-512)** de notre cluster **Amazon EKS**.
> 5. **15 ms** pour l'évaluation des règles métiers du marchand et le retour de la décision (`ALLOW`, `BLOCK` ou `CHALLENGE_3DS`).
>
> En tâche de fond, **Evidently AI** surveille chaque jour la dérive des données via l'indice **PSI (*Population Stability Index*)** : si le **PSI dépasse 0,20**, Airflow déclenche un réentraînement GPU Spot sur **Amazon SageMaker**, suivi d'un déploiement **Shadow / Canary de 48 heures** avant bascule en production. »

- 👉 **[Passer à la Slide 5]**

🔒 SLIDE 5 — 4. Haute Disponibilité (RPO=0) & Sécurité PCI-DSS v4.0 / RGPD *(⏱ 03:20 – 04:15 | 55 sec)*

- **Support à l'écran :** *Slide 5 (* `PRESENTATION_STRIPE_BLOC2.pptx` *— Livrable* `06` *)*

> « Sur la **Slide 5**, l'architecture garantit une résilience et une conformité bancaire de niveau 1 :
>
> - **Continuité d'activité :** Aurora PostgreSQL réplique chaque écriture de façon synchrone en **6 copies sur 3 Zones de Disponibilité (Quorum 4 sur 6)** à Paris, garantissant un **RPO égal à zéro** (aucune transaction perdue) et un **RTO inférieur à 60 secondes**.
> - **Conformité PCI-DSS v4.0 Niveau 1 :** Dès l'Edge, le numéro de carte (*PAN*) et le CVV sont interceptés par un **Tokenization Vault** isolé sur **AWS CloudHSM**, qui renvoie uniquement un

> jeton opaque (`pm_...`) et les 4 derniers chiffres (`card_last4`). **Zéro numéro de carte en clair** ne traverse nos bases SQL, NoSQL ou Kafka.
>
> - **Conformité RGPD :** Chiffrement intégral en **TLS 1.3** en transit et **AES-256 via AWS KMS (CMK)** au repos, gestion dynamique des secrets via **AWS Secrets Manager (IRSA)**, pseudonymisation de l'email en **SHA-256 avec sel**, et orchestrateur **Airflow + Spark** pour le droit à l'oubli (anonymisation `DELETED_USER` en OLTP tout en conservant les écritures comptables légales 10 ans, et purge dans DocumentDB et S3). »

- 👉 **[Passer à la Slide 6]**

💰 SLIDE 6 — 5. Requêtes Métier SQL/NoSQL, FinOps & GreenOps *(⏱️ 04:15 – 05:00 | 45 sec)*

- **Support à l'écran :** *Slide 6 (`PRESENTATION_STRIPE_BLOC2.pptx` — Livrables `06` & `08`)*

> « Enfin, sur la **Slide 6**, j'ai validé l'exploitation du modèle (Livrable 08) avec **4 requêtes SQL OLAP** (classement GPV et marge nette Stripe, taux de refus > 8 %, surveillance du seuil critique de **1 % de Chargebacks Visa/Mastercard**, et analyse de cohorte du *Churn* marchand) ainsi que **3 pipelines d'agrégation NoSQL** (`$match` , `$unwind` , `$group`) détectant en direct les attaques de *Carding*, les *Bots* (saisie CVV `< 200 ms`) et les pannes de webhooks.
>
> Côté **FinOps et GreenOps**, l'infrastructure complète pour 15 millions de transactions par jour est chiffrée à **25 000 $ par mois** sur AWS Pricing Calculator, optimisée par trois leviers : **RDS Savings Plans sur 3 ans (-38 %)** sur Aurora, **instances GPU Spot (-70 %)** sur SageMaker uniquement pendant les 8 heures d'entraînement hebdomadaire, et hébergement bas-carbone à **Paris (`eu-west-3` , ~50 g CO_2e/kWh)** avec archivage **Parquet Snappy / S3 Glacier**.
>
> Je vous remercie pour votre attention, et je suis prêt à répondre à toutes vos questions. »

🛡️ TOP 5 QUESTIONS RÉFLEXES DU JURY APRÈS CE SPEECH (RÉPONSES EN 25s)

1. **« Pourquoi avoir choisi ONNX Runtime sur CPU (`c6i` AVX-512) plutôt que des GPU pour l'inférence temps réel de Stripe Radar ? »**

   > *« Parce que sur des données tabulaires avec un modèle d'arbres de décision (XGBoost / LightGBM) et une taille de batch unitaire (requête par requête en temps réel), l'aller-retour PCIe entre la RAM CPU et la VRAM d'un GPU ajoute de la latence et multiplie le coût par 5. Compilé en C++ dans ONNX Runtime avec les instructions vectorielles AVX-512 de nos instances `c6i.2xlarge` , l'arbre s'exécute en 30 ms pour 3 800 $/mois sur 3 AZs. Nous réservons les GPU (`ml.g5.2xlarge` en Spot à -70 %) uniquement au réentraînement batch hebdomadaire sur SageMaker. »*

2. **« Pourquoi stocker tous les montants en `BIGINT` en centimes dans Aurora et Redshift plutôt qu'en `FLOAT` ou `DECIMAL` ? »**

   > *« En informatique financière, les types à virgule flottante (`FLOAT` / `DOUBLE`) utilisent la représentation binaire IEEE 754 qui provoque des erreurs d'arrondi (par exemple `0.1 + 0.2 = 0.30000000000000004`). Sur 15 millions de transactions par jour, cela fausserait la comptabilité de milliers d'euros. En stockant chaque montant en centimes sous forme d'entier 64 bits (`BIGINT`) (ex : `4999` pour 49,99 €), les additions et soustractions CPU sont 100 % exactes et ultra-rapides. »*

3. **« Pourquoi utiliser Debezium (CDC sur le WAL) plutôt qu'une écriture double (Dual Write) ou un `SELECT` périodique ? »**

   > *« Un `SELECT` périodique surcharge le CPU d'Aurora et rate les suppressions intermédiaires. Quant au* Dual Write *(écrire dans Aurora puis dans Kafka depuis l'API), si le réseau coupe entre les deux appels, les bases deviennent incohérentes. Avec Debezium CDC, l'API fait un seul commit ACID dans Aurora, et Debezium lit de façon asynchrone le journal binaire WAL (*Write-Ahead Log*) au niveau fichier avec zéro impact de performance, puis publie dans Kafka en Avro compressé. »*

4. **« Comment conciliez-vous le Droit à l'Oubli (Article 17 RGPD) avec l'obligation légale de conserver les transactions comptables pendant 10 ans ? »**

   > *« Comme détaillé dans mon livrable 06, le RGPD (Art. 17.3.b) prévoit une exception explicite lorsque le traitement est requis par une obligation légale comptable et fiscale (Code de commerce : 10 ans). Notre orchestrateur Airflow applique donc une anonymisation sélective : dans `Customer`, toutes les données personnelles (PII) sont écrasées par `DELETED_USER_XXXX` et les traces matérielles dans DocumentDB sont purgées, tandis que les lignes de facturation dans `Transaction` et `S3 Glacier` sont conservées uniquement rattachées à cet identifiant anonyme irréversible. »*

5. **« Pourquoi avoir choisi l'imbrication (*Embedding*) pour `hardware_telemetry` mais le référencement (*Referencing*) pour `merchant_id` dans Amazon DocumentDB ? »**

   > *« C'est la règle d'or de la modélisation documentaire NoSQL (Livrable 04) : les attributs matériels (`hardware_telemetry` et les 10 dernières IPs) sont bornés et toujours lus ensemble avec l'empreinte lors du scoring, donc l'*Embedding *permet de tout récupérer en une seule lecture disque $O(1)$ (`findOne`) sans jointure. À l'inverse, un marchand génère des millions d'événements (cardinalité non bornée) : l'imbriquer exploserait la limite BSON de 16 Mo par document, d'où l'utilisation du* Referencing *par `merchant_id`. »*